GUÍA «PERMISOS EN LINUX (LPIC-1)»

Manual del laboratorio

Usuarios **ana · beto · carla**, grupos **equipo · soporte** y la carpeta **~/lab-permisos**: cómo montarlo, usarlo y borrarlo.

Edición de 16/10/2026 · github.com/TheBillBull/Guias-y-Herramientas

> **Guía «Permisos en Linux (LPIC-1)»** · Este documento explica el laboratorio que acompaña a la guía: qué crea, cómo se monta, cómo se usa para probar permisos *como otras personas*, cómo se reinicia y cómo se borra sin dejar rastro. Al final (anexos) tienes el **código completo** de los scripts.

---

1 · ¿Para qué sirve?

Para entender un permiso **hay que ver cómo falla**. Y para eso hacen falta **otras personas**: alguien que sea del grupo, alguien que no lo sea, alguien que sea el dueño. El laboratorio te da, en tu propia máquina Ubuntu:

- una carpeta de práctica, `~/lab-permisos`, con ficheros de ejemplo (incluidos los tres *scripts* de la captura de pantalla: `cerve.sh`, `saludos.sh` y `salario.sh`, con **los mismos tamaños, fechas y permisos**);
- tres **usuarios de práctica** (`ana`, `beto`, `carla`) y dos **grupos** (`equipo`, `soporte`), para ejecutar órdenes **disfrazado de ellos** con `sudo -u`.

Todas las salidas que ves en la guía se obtuvieron con este mismo laboratorio.

2 · Qué crea… y qué NO toca

El script…	Detalle
Crea la carpeta `~/lab-permisos`	con `cerve.sh`, `saludos.sh`, `salario.sh` y la carpeta `datos/` (`agenda.txt`, `notas.txt`, `secreto.txt`, `ventas.csv`)
Crea 3 usuarios	`ana`, `beto`, `carla`; con shell `/bin/bash`, su carpeta personal y **sin contraseña** (cuenta bloqueada: nadie puede entrar con ellos; solo los usas con `sudo -u`)
Crea 2 grupos	`equipo` (ana, beto) y `soporte` (beto, carla)
Cambia un permiso	añade `x` a «otros» en tu carpeta personal (`chmod o+x ~`; en Ubuntu 24.04 pasa de `750` a `751`) para que los usuarios de práctica puedan *atravesarla* hasta `~/lab-permisos`. Lo explica el capítulo 3 de la guía
NO toca	contraseñas, `/etc/sudoers`, red, servicios, tus ficheros ni tus usuarios. Nunca da privilegios de `sudo` a nadie

Los usuarios que crea llevan en su campo de comentario la marca `lab-permisos-lpic1`; `--limpiar` **solo borra usuarios con esa marca**.

3 · Requisitos

- **Ubuntu** 22.04 o 24.04 (vale cualquier distribución con GNU coreutils y `sudo`).
- Un usuario normal **que pueda usar** `sudo` (es el que se instala en el asistente de Ubuntu).
- `bash`, `git` (para descargar el repositorio; si no lo tienes: `sudo apt install git`).

> 🛑 **No ejecutes el script como** `root` (con `sudo bash …`): se negará. Hazlo con tu usuario normal; él mismo pedirá `sudo` solo para crear los usuarios.

4 · Puesta en marcha, paso a paso

```
# 1. Descarga la guía y el laboratorio
cd ~
git clone https://github.com/TheBillBull/Guias-y-Herramientas.git

# 2. Monta el laboratorio (te pedirá tu contraseña de sudo y que confirmes)
bash ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/laboratorio/preparar-laboratorio.sh

# 3. Entra y mira
cd ~/lab-permisos
ls -l
```

Esto es lo que verás (los colores y las fechas pueden variar):

```
Tu carpeta personal era 750; ahora es 751 (los demás pueden atravesarla, no listarla). Se explica en el capítulo 3.
✔ Laboratorio creado en /home/alumno/lab-permisos
Voy a crear (con sudo) los usuarios ana, beto y carla y los grupos equipo y soporte.
Se pueden borrar después con:  bash …/preparar-laboratorio.sh --limpiar
¿Continuar? [s/N] s
✔ Usuarios: ana (equipo) · beto (equipo, soporte) · carla (soporte)

Listo. Ahora:  cd ~/lab-permisos
```

Sin `git`: copia el código del **Anexo A** de este documento a un fichero `preparar-laboratorio.sh` y ejecútalo con `bash preparar-laboratorio.sh` (comprueba antes la copia con `bash -n preparar-laboratorio.sh`: no debe imprimir nada).

5 · Opciones del script

Orden	Qué hace
`bash preparar-laboratorio.sh`	crea el laboratorio completo (carpeta + usuarios + grupos)
`… --sin-usuarios`	solo la carpeta `~/lab-permisos` (no necesita `sudo`)
`… --solo-usuarios`	solo los usuarios y grupos
`… --si`	no pregunta antes de crear los usuarios
`… --limpiar`	**borra todo lo que creó el script** (carpeta, usuarios, grupos y el permiso de tu carpeta personal)
`… --ayuda`	muestra la ayuda

Es **seguro repetirlo**: cada ejecución deja la carpeta como nueva (sirve para empezar de cero cuando un ejercicio salga mal).

6 · El elenco

Quién	Qué es	Grupos
🧑‍🎓 **alumno** (tú)	el usuario con el que trabajas	`alumno`
👩 **ana**	compañera de equipo	`ana`, `equipo`
👨 **beto**	de equipo y de soporte	`beto`, `equipo`, `soporte`
👩‍🔧 **carla**	solo de soporte	`carla`, `soporte`

Para comprobarlo:

```
$ id ana
uid=1002(ana) gid=1002(ana) groups=1002(ana),1005(equipo)
$ id beto
uid=1003(beto) gid=1003(beto) groups=1003(beto),1005(equipo),1006(soporte)
$ id carla
uid=1004(carla) gid=1004(carla) groups=1004(carla),1006(soporte)
$ getent group equipo soporte
equipo:x:1005:ana,beto
soporte:x:1006:beto,carla
```

Los números (UID/GID) serán otros en tu máquina; lo que importa son los **nombres** y los grupos. `alumno` **no** pertenece a `equipo` ni a `soporte`: eso da mucho juego en los ejercicios.

7 · Lo que hay en `~/lab-permisos`

```
$ cd ~/lab-permisos
$ ls -l
total 16
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
drwxr-xr-x 2 alumno alumno 4096 Oct 16 17:08 datos
-rwxrwxr-x 1 alumno alumno  419 Oct 14 08:43 salario.sh
-rwxr-x--x 1 alumno alumno   93 Oct 13 08:07 saludos.sh
$ ls -l datos
total 16
-rw-r--r-- 1 alumno alumno 56 Oct 12 09:30 agenda.txt
-rw-r--r-- 1 alumno alumno 39 Oct 12 09:30 notas.txt
-rw------- 1 alumno alumno 44 Oct 12 09:30 secreto.txt
-rw-r--r-- 1 alumno alumno 37 Oct 12 09:30 ventas.csv
```

Fichero	Permisos	Para qué se usa en la guía
`cerve.sh`	`775` (`-rwxrwxr-x`), 1015 bytes	el ejemplo del **capítulo 1**: ¡es tu línea de `ls -l`!
`saludos.sh`	`751` (`-rwxr-x--x`), 93 bytes	un script que «otros» **no** pueden ejecutar (les falta la `r`)
`salario.sh`	`775`, 419 bytes	el de tu captura
`datos/secreto.txt`	`600`	para ver cómo `carla` recibe *Permission denied*
`datos/agenda.txt`, `notas.txt`, `ventas.csv`	`644`	ficheros normales de lectura pública

8 · Probar permisos «como otra persona»

La técnica estrella de toda la guía es `sudo -u USUARIO COMANDO`: ejecuta el comando *siendo* ese usuario. Por ejemplo, ¿puede `carla` leer el fichero secreto?

```
$ sudo -u carla cat datos/secreto.txt
cat: datos/secreto.txt: Permission denied
$ cat datos/secreto.txt
La contraseña del wifi es: patata-frita-42
```

Quiero saber…	Comando
¿Puede `ana` **leer** `f`?	`sudo -u ana cat f`
¿Puede `ana` **escribir** en `f`?	`sudo -u ana sh -c 'echo x >> f'`
¿Puede `ana` **ejecutar** `f`?	`sudo -u ana ./f`
¿Quién soy ahora?	`sudo -u ana id`
¿Puede `ana` **entrar** en la carpeta `d`?	`sudo -u ana ls d`

La «radiografía» de un fichero: la función `quien`

Pega esta función en tu terminal y te dice, de un vistazo, **qué pueden hacer los cuatro usuarios** con cualquier fichero:

```
quien() {
  printf '%-8s leer  escribir  ejecutar\n' "$1"
  for u in alumno ana beto carla; do
    printf '%-8s' "$u"
    sudo -u $u test -r "$1" && printf '  ✔      ' || printf '  ✘      '
    sudo -u $u test -w "$1" && printf '  ✔       ' || printf '  ✘       '
    sudo -u $u test -x "$1" && printf '  ✔' || printf '  ✘'
    echo
  done
}
```

```
$ cp saludos.sh f ; sudo chgrp equipo f ; chmod 640 f
$ quien f
f        leer  escribir  ejecutar
alumno    ✔       ✔        ✘
ana       ✔       ✘        ✘
beto      ✔       ✘        ✘
carla     ✘       ✘        ✘
```

Si algo no cuadra: `diagnostica`

La herramienta `diagnostica` (anexo C) recorre la ruta tramo a tramo con el algoritmo real de Linux y señala **qué carpeta o fichero bloquea** a un usuario:

```
$ sudo diagnostica -u carla /home/alumno/lab-permisos/datos/secreto.txt
```

9 · Reiniciar, limpiar y desinstalar

Situación	Qué hacer
Un ejercicio ha dejado el laboratorio hecho un lío	volver a ejecutar `bash …/preparar-laboratorio.sh` (la carpeta queda **como nueva**)
Quieres empezar un capítulo «limpio»	lo mismo
Has terminado la guía y quieres **borrarlo todo**	`bash …/preparar-laboratorio.sh --limpiar`

`--limpiar`:

1. borra `~/lab-permisos` (¡cualquier cosa tuya que hayas guardado ahí desaparece!);
2. borra los usuarios `ana`, `beto`, `carla` **solo si llevan la marca** del laboratorio, con sus carpetas personales;
3. borra los grupos `equipo` y `soporte` **solo si todos sus miembros son de la guía**;
4. devuelve a tu carpeta personal su permiso original (`750`).

10 · Garantías de seguridad del script

- **No toca lo que no es suyo:** si ya existe un usuario `ana` (o `beto`, `carla`) **sin la marca** del laboratorio, o un grupo `equipo`/`soporte` que ya tenías con otros miembros, el script **se niega a continuar** y **no cambia nada**.
- **Valida antes de crear:** comprueba todos los conflictos **antes** de crear ningún usuario o grupo (nunca deja un estado a medias).
- **Rutas peligrosas:** se niega a trabajar sobre `/`, sobre tu carpeta personal entera o sobre una ruta vacía.
- **Sin contraseñas ni `sudoers`:** los usuarios de práctica no pueden iniciar sesión y no tienen ningún privilegio.
- **Reversible:** `--limpiar` deshace todo lo que hizo (incluido el `chmod o+x ~`).

11 · Problemas frecuentes

Síntoma	Causa y solución
`No ejecutes la creación de ~/lab-permisos como root`	lo lanzaste con `sudo`. Ejecútalo con tu usuario normal
`Ya existe un usuario 'ana' que NO es del laboratorio`	tienes un usuario con ese nombre que no creó el script. Renómbralo o usa una máquina limpia; el script no lo tocará
`Ya existe un grupo 'equipo' que NO es de la guía`	igual con el grupo. Cámbiale el nombre o bórralo si no lo usas
`sudo: command not found` / *no estás en el fichero sudoers*	tu usuario no es administrador. Usa uno que lo sea, o ejecuta con `--sin-usuarios` (los ejercicios con otros usuarios no funcionarán)
`ana` recibe *Permission denied* en **todo**, incluso en lo que debería poder leer	falta el permiso `x` de «otros» en tu carpeta personal. Vuelve a ejecutar el script (o `chmod o+x ~`)
No puedo borrar `~/lab-permisos` (*Operation not permitted*)	algún ejercicio dejó ficheros **inmutables** (`chattr +i`). Ejecuta `sudo chattr -R -i ~/lab-permisos` y repite `--limpiar`
Mis salidas no coinciden con el libro	es normal: UID, GID, fechas, tamaños de ficheros del sistema y el idioma de los mensajes cambian. Los **permisos** y los **nombres** deben coincidir
Me pide la contraseña de `sudo` muchas veces	`sudo` la olvida a los pocos minutos; es normal
Los meses salen en minúscula (`oct`) y los errores en español	tu sistema está en español; la guía se generó en inglés. Equivalencias en el capítulo 0

12 · Las herramientas que acompañan al laboratorio

Se instalan una sola vez:

```
sudo install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/permisos    /usr/local/bin/
sudo install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/diagnostica /usr/local/bin/
```

Herramienta	Para qué	Ejemplos
`permisos`	traduce octal ⇄ letras ⇄ binario ⇄ decimal ⇄ hex; calcula `umask`; aplica expresiones de `chmod`	`permisos 755` · `permisos -rwsr-xr-x` · `permisos 644 g+w,o-r` · `permisos -u 027`
`diagnostica`	explica **por qué** un usuario puede o no acceder a una ruta	`sudo diagnostica -u carla /ruta/fichero`
`calculadora-chmod.html`	calculadora **visual** de `chmod` (se abre en el navegador)	casillas `r/w/x` y bits especiales
`entrenador-conversion.html`	**entrenador de velocidad** de conversiones	sprint de 60 s

Ejemplo de `permisos`:

```
$ permisos 4755
4755

  Octal     4755
  Letras    rwsr-xr-x      (en ls -l: -rwsr-xr-x)
  Binario   100 111 101 101      (3 bits especiales + 3 × 3)
  Decimal   2541
  Hex       0x9ed

  dueño   rws  (7)  leer, escribir, ejecutar
  grupo   r-x  (5)  leer, ejecutar
  otros   r-x  (5)  leer, ejecutar

  Bits especiales:
    · SUID (4)    se ejecuta con los privilegios del DUEÑO

  chmod 4755 FICHERO
  chmod u=rwx,g=rx,o=rx,u+s FICHERO
```

Anexo A · `preparar-laboratorio.sh`

El script que crea y borra el laboratorio. Cópialo a un fichero y ejecútalo con `bash preparar-laboratorio.sh` (como usuario normal).

```
#!/usr/bin/env bash
# ============================================================================
#  preparar-laboratorio.sh  ·  Laboratorio de la guía "Permisos en Linux (LPIC-1)"
#
#  Qué hace (y SOLO esto):
#    1. Crea la carpeta  ~/lab-permisos  con unos ficheros de ejemplo.
#    2. Crea 3 usuarios de práctica (ana, beto, carla) y 2 grupos (equipo, soporte)
#       para poder probar "qué pasa cuando OTRA persona intenta leer mi fichero".
#       Esto necesita sudo (te pedirá tu contraseña).
#    3. Le da a tu carpeta personal el permiso "x" para los demás (chmod o+x ~),
#       porque en Ubuntu 24.04 tu carpeta personal es 750 y ana/beto/carla no
#       podrían ni entrar. (Lo explica el capítulo 3.)
#
#  Uso:
#    bash preparar-laboratorio.sh              crea el laboratorio completo
#    bash preparar-laboratorio.sh --sin-usuarios   solo la carpeta (sin sudo)
#    bash preparar-laboratorio.sh --solo-usuarios  solo los usuarios y grupos
#    bash preparar-laboratorio.sh --limpiar        BORRA todo lo que creó el script
#    bash preparar-laboratorio.sh --si             no pregunta antes de crear usuarios
#
#  Es seguro repetirlo: cada vez deja el laboratorio como nuevo.
#  Los usuarios que crea llevan en su "comentario" la marca "lab-permisos-lpic1";
#  --limpiar SOLO borra usuarios que tengan esa marca (nunca toca los tuyos).
# ============================================================================

set -u

LAB="${LAB_DIR:-$HOME/lab-permisos}"
MARCA="lab-permisos-lpic1"
USUARIOS=(ana beto carla)
GRUPOS=(equipo soporte)
FICHERO_MODO_HOME="$HOME/.lab-permisos-modo-home"

# Seguridad: nunca trabajamos sobre "/", sobre tu carpeta personal ni sobre una ruta vacía
case "$LAB" in
  ""|/|"$HOME"|"$HOME"/) echo "Ruta de laboratorio peligrosa ('$LAB'): me niego a seguir."; exit 1 ;;
esac

SUD=()
[ "$(id -u)" -eq 0 ] || SUD=(sudo)

OPCION_USUARIOS=1
OPCION_LAB=1
OPCION_LIMPIAR=0
PREGUNTAR=1

for arg in "$@"; do
  case "$arg" in
    --sin-usuarios)  OPCION_USUARIOS=0 ;;
    --solo-usuarios) OPCION_LAB=0 ;;
    --limpiar)       OPCION_LIMPIAR=1 ;;
    --si)            PREGUNTAR=0 ;;
    -h|--ayuda|--help) sed -n '2,24p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
    *) echo "Opción desconocida: $arg (prueba --ayuda)"; exit 1 ;;
  esac
done

aviso() { printf '\033[1;33m%s\033[0m\n' "$*"; }

# ¿Es el grupo "nuestro"? Sí si NO existe, o si existe y TODOS sus miembros son usuarios de la guía
# (un grupo que ya tuvieras tú, vacío o con otras personas, no se toca nunca).
grupo_es_nuestro() {
  local g=$1 miembros m u ok
  getent group "$g" >/dev/null || return 0
  miembros=$(getent group "$g" | cut -d: -f4)
  [ -n "$miembros" ] || return 1
  IFS=',' read -ra arr <<<"$miembros"
  for m in "${arr[@]}"; do
    ok=0
    for u in "${USUARIOS[@]}"; do [ "$m" = "$u" ] && ok=1; done
    [ "$ok" -eq 1 ] || return 1
  done
  return 0
}
ok()    { printf '\033[1;32m✔\033[0m %s\n' "$*"; }

# ---------------------------------------------------------------- limpiar ----
limpiar() {
  echo "Limpiando el laboratorio…"
  if [ -e "$LAB" ]; then
    chmod -R u+rwX "$LAB" 2>/dev/null
    rm -rf "$LAB" 2>/dev/null || "${SUD[@]}" rm -rf "$LAB"
    ok "Borrada la carpeta $LAB"
```

```
  fi
  # (decidimos qué grupos son nuestros ANTES de borrar a sus miembros)
  borrar_grupos=()
  for g in "${GRUPOS[@]}"; do
    if getent group "$g" >/dev/null; then
      if grupo_es_nuestro "$g"; then borrar_grupos+=("$g"); else aviso "El grupo '$g' no es de la guía (tiene otros miembros o ya
existía): no lo toco."; fi
    fi
  done
  for u in "${USUARIOS[@]}"; do
    if getent passwd "$u" >/dev/null && getent passwd "$u" | grep -q ":$MARCA"; then
      "${SUD[@]}" userdel -r "$u" 2>/dev/null
      ok "Borrado el usuario $u"
    fi
  done
  for g in "${borrar_grupos[@]}"; do
    "${SUD[@]}" groupdel "$g" 2>/dev/null && ok "Borrado el grupo $g"
  done
  if [ -f "$FICHERO_MODO_HOME" ]; then
    chmod "$(cat "$FICHERO_MODO_HOME")" "$HOME" && ok "Restaurado el permiso original de $HOME ($(cat "$FICHERO_MODO_HOME"))"
    rm -f "$FICHERO_MODO_HOME"
  fi
}

# ------------------------------------------------------------- usuarios ----
crear_usuarios() {
  if ! command -v sudo >/dev/null && [ "$(id -u)" -ne 0 ]; then
    aviso "No encuentro sudo: sin él no puedo crear los usuarios de práctica."
    return 1
  fi
  if [ "$PREGUNTAR" -eq 1 ]; then
    echo "Voy a crear (con sudo) los usuarios ana, beto y carla y los grupos equipo y soporte."
    echo "Se pueden borrar después con:  bash $0 --limpiar"
    read -r -p "¿Continuar? [s/N] " r
    case "$r" in s|S|si|SI|sí|Sí) ;; *) echo "Vale, no creo los usuarios."; return 1 ;; esac
  fi

  # 1) VALIDAR todo antes de tocar nada (así nunca queda un estado a medias)
  for g in "${GRUPOS[@]}"; do
    if ! grupo_es_nuestro "$g"; then
      aviso "Ya existe un grupo '$g' que NO es de la guía. No lo toco; sin él los ejercicios de grupos no funcionarán."
      return 1
    fi
  done
  for u in "${USUARIOS[@]}"; do
    if getent passwd "$u" >/dev/null && ! getent passwd "$u" | grep -q ":$MARCA"; then
      aviso "Ya existe un usuario '$u' que NO es del laboratorio. No lo toco, pero los ejercicios esperan otro 'ana/beto/carla'."
      return 1
    fi
  done

  # 2) CREAR
  for u in "${USUARIOS[@]}"; do
    getent passwd "$u" >/dev/null || "${SUD[@]}" useradd -m -s /bin/bash -c "$MARCA" "$u"
  done
  for g in "${GRUPOS[@]}"; do
    getent group "$g" >/dev/null || "${SUD[@]}" groupadd "$g"
  done
  "${SUD[@]}" usermod -aG equipo ana
  "${SUD[@]}" usermod -aG equipo beto
  "${SUD[@]}" usermod -aG soporte beto
  "${SUD[@]}" usermod -aG soporte carla
  ok "Usuarios: ana (equipo) · beto (equipo, soporte) · carla (soporte)"
}

# ------------------------------------------------------------- laboratorio ----
crear_lab() {
  if [ "$(id -u)" -eq 0 ]; then
    echo "No ejecutes la creación de ~/lab-permisos como root: hazlo con tu usuario normal."
    exit 1
  fi
  if [ -e "$LAB" ]; then
    chmod -R u+rwX "$LAB" 2>/dev/null
    rm -rf "$LAB" 2>/dev/null || { aviso "Hay ficheros de otros usuarios; uso sudo para borrarlos"; sudo rm -rf "$LAB"; }
  fi
  mkdir -p "$LAB/datos" && cd "$LAB" || exit 1

  # --- los tres scripts de la captura de pantalla (mismos permisos) ----------
  # (Los tres miden EXACTAMENTE 1015, 93 y 419 bytes, como en la captura.)
  cat > cerve.sh <<'EOF'
#!/bin/bash
# cerve.sh · el menú de la cervecería (script de prueba para practicar permisos)
#
# Cada línea que empieza por # es un comentario: bash la ignora.
# Este fichero necesita permiso de EJECUCIÓN (x) para poder lanzarlo con ./cerve.sh
# y permiso de LECTURA (r) para que bash pueda leer lo que hay dentro.

# Ojo: el menu es de broma; lo importante son los permisos, no la cerveza.
nombre=$(whoami)
dia=$(date +%A)

echo "============================="
```

```
echo "  CERVECERÍA «EL PERMISO»"
echo "============================="
echo "Bienvenido, $nombre. Hoy es $dia."
echo

case "$1" in
  rubia)  echo "Una rubia bien fría. ¡Salud!" ;;
  negra)  echo "Una negra cremosa. ¡Salud!" ;;
  tostada) echo "Una tostada de abadía. ¡Salud!" ;;
  sin)    echo "Una sin alcohol, que mañana hay clase." ;;
  *)      echo "Uso: ./cerve.sh rubia|negra|tostada|sin" ;;
esac

# Fin del script. Cuando termines de leerlo, prueba a quitarle el permiso x
# con:  chmod -x cerve.sh   y mira qué pasa al lanzarlo.
EOF
  cat > saludos.sh <<'EOF'
#!/bin/bash
# saludos.sh · dice hola y te cuenta quién eres, amigo.
echo "Hola, $(whoami)"
EOF
  cat > salario.sh <<'EOF'
#!/bin/bash
# salario.sh · calcula un sueldo anual (ficticio) con 14 pagas
# Uso:  ./salario.sh 1800     (si no das nada, usa 1800 al mes)

# Ojo: los numeros son inventados, aqui solo practicamos con chmod y umask
mensual=${1:-1800}
pagas=14

anual=$((mensual * pagas))
mes_neto=$((mensual * 80 / 100))

echo "Bruto mensual : $mensual euros"
echo "Bruto anual   : $anual euros"
echo "Neto aprox/mes: $mes_neto euros"
EOF
  chmod 775 cerve.sh;  touch -d '2026-10-09 16:16' cerve.sh
  chmod 751 saludos.sh; touch -d '2026-10-13 08:07' saludos.sh
  chmod 775 salario.sh; touch -d '2026-10-14 08:43' salario.sh

  # --- ficheros de datos con permisos variados ------------------------------
  printf 'Reunión el lunes a las 10:00\nLlevar el informe impreso\n' > datos/agenda.txt
  printf 'Cliente;Importe\nAcme;1200\nGlobex;860\n'                  > datos/ventas.csv
  printf 'La contraseña del wifi es: patata-frita-42\n'              > datos/secreto.txt
  printf 'Notas de clase: chmod, chown, umask…\n'                    > datos/notas.txt
  chmod 644 datos/agenda.txt datos/ventas.csv datos/notas.txt
  chmod 600 datos/secreto.txt
  touch -d '2026-10-12 09:30' datos/*.txt datos/ventas.csv

  # --- carpeta personal: que ana/beto/carla puedan "pasar" hasta el laboratorio
  modo_home=$(stat -c %a "$HOME")
  if [ "$(( 8#$modo_home & 8#001 ))" -eq 0 ]; then
    echo "$modo_home" > "$FICHERO_MODO_HOME"
    chmod o+x "$HOME"
    aviso "Tu carpeta personal era $modo_home; ahora es $(stat -c %a "$HOME") (los demás pueden atravesarla, no listarla). Se explica
en el capítulo 3."
  fi
  ok "Laboratorio creado en $LAB"
}

# ---------------------------------------------------------------------- main ----
if [ "$OPCION_LIMPIAR" -eq 1 ]; then limpiar; exit 0; fi
[ "$OPCION_USUARIOS" -eq 1 ] || [ "$OPCION_LAB" -eq 1 ] || exit 0

if [ "$OPCION_LAB" -eq 1 ];      then crear_lab; fi
if [ "$OPCION_USUARIOS" -eq 1 ]; then crear_usuarios || aviso "Seguimos sin usuarios de práctica (los capítulos que los usan te lo
dirán)."; fi
echo
echo "Listo. Ahora:  cd ~/lab-permisos"
```

Anexo B · `permisos`

Traductor de permisos (octal, letras, binario, decimal, hex), `umask` y expresiones de `chmod`. Instálalo con `sudo install -m 755 permisos /usr/local/bin/`.

```
#!/usr/bin/env bash
# ============================================================================
#  permisos · traductor de permisos de Linux
#  octal ⇄ letras (ls -l) ⇄ binario ⇄ decimal ⇄ hexadecimal, umask y chmod
#
#  No necesita nada más que bash y las coreutils (chmod, stat, mktemp).
#  Las expresiones de chmod se evalúan con el chmod REAL, así que el resultado
#  es exactamente el que obtendrías de verdad.
# ============================================================================
LC_ALL=C
export LC_ALL

uso() {
cat <<'EOF'
permisos · traductor de permisos de Linux

USO
  permisos 755                octal (3 o 4 cifras: 755, 644, 4755, 1777…)
  permisos rwxr-xr-x          letras (9 caracteres; con s S t T si hay bits especiales)
  permisos -rwxr-xr-x         una línea de "ls -l" (el 1.er carácter es el tipo: - d l …)
  permisos 0b111101101        binario
  permisos 0x1ed              hexadecimal
  permisos d:493              decimal (el "d:" evita confundirlo con un octal)
  permisos fichero…           los permisos REALES de ficheros o carpetas
  permisos u=rwx,go=rx        expresión de chmod aplicada a 000
  permisos 644 g+w,o-r        modo base + expresión de chmod  →  resultado
  permisos f.sh u+x           (el modo base puede ser un fichero real)
  permisos -u 027             umask → permisos de los ficheros y carpetas nuevos
  permisos -u                 (sin número) usa tu umask actual
  permisos -d …               trata el modo como el de una CARPETA (afecta a la X de chmod)
  permisos -h                 esta ayuda

EJEMPLOS
  permisos 4755               ¿qué es un 4755?
  permisos -rwsr-xr-x         lo mismo, desde las letras
  permisos /etc/shadow /tmp   permisos reales
  permisos 640 u+x,g=rw       qué dejaría "chmod u+x,g=rw" sobre un 640
EOF
}

ES_DIR=0
BITS=(256 128 64 32 16 8 4 2 1)

error() { printf 'permisos: %s\n' "$*" >&2; exit 1; }

# entero (0..4095) → "rwxr-xr-x" (con s S t T)
letras() {
  local m=$1 s="" i
  local ch=(r w x r w x r w x)
  for i in 0 1 2 3 4 5 6 7 8; do
    if (( m & BITS[i] )); then s+=${ch[i]}; else s+=-; fi
  done
  if (( m & 04000 )); then if [[ ${s:2:1} == x ]]; then s=${s:0:2}s${s:3}; else s=${s:0:2}S${s:3}; fi; fi
  if (( m & 02000 )); then if [[ ${s:5:1} == x ]]; then s=${s:0:5}s${s:6}; else s=${s:0:5}S${s:6}; fi; fi
  if (( m & 01000 )); then if [[ ${s:8:1} == x ]]; then s=${s:0:8}t; else s=${s:0:8}T; fi; fi
  printf '%s' "$s"
}

# "rwxr-xr-x" → entero (devuelve 1 si no son letras válidas)
de_letras() {
  local s=$1 m=0 i c
  [[ $s =~ ^[-r][-w][-xsS][-r][-w][-xsS][-r][-w][-xtT]$ ]] || return 1
  for i in 0 1 2 3 4 5 6 7 8; do
    c=${s:i:1}
    case $c in
      r|w|x) (( m |= BITS[i] )) ;;
      s) (( m |= BITS[i] )); if (( i == 2 )); then (( m |= 04000 )); else (( m |= 02000 )); fi ;;
      S) if (( i == 2 )); then (( m |= 04000 )); else (( m |= 02000 )); fi ;;
      t) (( m |= BITS[i] | 01000 )) ;;
      T) (( m |= 01000 )) ;;
    esac
  done
  printf '%d' "$m"
}

# los n bits de v en binario, en grupos de 3
binario() {
  local v=$1 n=$2 out="" i
  for (( i=n-1; i>=0; i-- )); do
    out+=$(( (v >> i) & 1 ))
    if (( i % 3 == 0 && i > 0 )); then out+=" "; fi
  done
  printf '%s' "$out"
```

```
}

# 0..7 → "leer, escribir…"
acciones() {
  local d=$1 dir=$2 t=() out="" x
  if (( dir )); then
    (( d & 4 )) && t+=("listar")
    (( d & 2 )) && t+=("crear/borrar/renombrar dentro")
    (( d & 1 )) && t+=("entrar")
  else
    (( d & 4 )) && t+=("leer")
    (( d & 2 )) && t+=("escribir")
    (( d & 1 )) && t+=("ejecutar")
  fi
  (( ${#t[@]} )) || { printf 'nada'; return; }
  for x in "${t[@]}"; do out+="${out:+, }$x"; done
  printf '%s' "$out"
}

# modo → u=rwx,g=rx,o=rx (solo la parte rwx)
simbolico() {
  local m=$1 out="" c d i
  local nom=(u g o)
  for i in 0 1 2; do
    d=$(( (m >> (6 - 3*i)) & 7 ))
    c=""
    (( d & 4 )) && c+=r
    (( d & 2 )) && c+=w
    (( d & 1 )) && c+=x
    out+="${nom[i]}=$c"
    (( i < 2 )) && out+=","
  done
  printf '%s' "$out"
}

# ------------------------------------------------------------ presentación --
mostrar() {
  local m=$1 dir=$2 titulo=${3:-}
  local esp=$(( (m >> 9) & 7 ))
  local u=$(( (m >> 6) & 7 )) g=$(( (m >> 3) & 7 )) o=$(( m & 7 ))
  local oct L tipo='-'
  if (( esp )); then oct=$(printf '%04o' "$m"); else oct=$(printf '%03o' "$m"); fi
  L=$(letras "$m")
  (( dir )) && tipo='d'
  [[ -n $titulo ]] && printf '%s\n\n' "$titulo"
  printf '  Octal     %s\n' "$oct"
  printf '  Letras    %s      (en ls -l: %s%s)\n' "$L" "$tipo" "$L"
  if (( esp )); then
    printf '  Binario   %s      (3 bits especiales + 3 × 3)\n' "$(binario "$m" 12)"
  else
    printf '  Binario   %s\n' "$(binario "$m" 9)"
  fi
  printf '  Decimal   %d\n' "$m"
  printf '  Hex       0x%x\n' "$m"
  printf '\n'
  printf '  dueño   %s  (%d)  %s\n' "${L:0:3}" "$u" "$(acciones "$u" "$dir")"
  printf '  grupo   %s  (%d)  %s\n' "${L:3:3}" "$g" "$(acciones "$g" "$dir")"
  printf '  otros   %s  (%d)  %s\n' "${L:6:3}" "$o" "$(acciones "$o" "$dir")"
  local sp=""
  if (( esp )); then
    printf '\n  Bits especiales:\n'
    if (( esp & 4 )); then
      printf '    · SUID (4)    se ejecuta con los privilegios del DUEÑO\n'; sp+=",u+s"
    fi
    if (( esp & 2 )); then
      if (( dir )); then printf '    · SGID (2)    lo que se cree dentro hereda el GRUPO de la carpeta\n'
      else printf '    · SGID (2)    se ejecuta con los privilegios del GRUPO\n'; fi
      sp+=",g+s"
    fi
    if (( esp & 1 )); then
      if (( dir )); then printf '    · sticky (1)  solo el dueño de cada fichero puede borrarlo\n'
      else printf '    · sticky (1)  (sin efecto en ficheros normales en Linux)\n'; fi
      sp+=",+t"
    fi
  fi
  printf '\n  chmod %s FICHERO\n' "$oct"
  printf '  chmod %s%s FICHERO\n' "$(simbolico "$m")" "$sp"
}

# ------------------------------------------------------- aplicar a chmod ----
aplicar() {  # base(entero) expresión dir → entero
  local base=$1 expr=$2 dir=$3 t f r
  t=$(mktemp -d) || error "no puedo crear un directorio temporal"
  if (( dir )); then f="$t/d"; mkdir "$f"; else f="$t/f"; : > "$f"; fi
  chmod "$(printf '%04o' "$base")" "$f" 2>/dev/null
  if ! chmod -- "$expr" "$f" 2>/dev/null; then rm -rf "$t"; return 1; fi
  r=$(stat -c %a "$f")
  rm -rf "$t"
  printf '%d' "$(( 8#$r ))"
}

es_expresion() { [[ $1 =~ ^[ugoa]*[-+=][rwxXstugo]*(,[ugoa]*[-+=][rwxXstugo]*)*$ ]]; }
```

```
# reconoce un modo; deja el entero en RES
reconocer() {
  local a=$1 s
  if [[ $a =~ ^[0-7]{1,4}$ ]]; then RES=$(( 8#$a )); return 0; fi
  if [[ $a =~ ^0[oO]([0-7]+)$ ]]; then RES=$(( 8#${BASH_REMATCH[1]} )); return 0; fi
  if [[ $a =~ ^0[bB]([01]+)$ ]]; then RES=$(( 2#${BASH_REMATCH[1]} )); return 0; fi
  if [[ $a =~ ^0[xX]([0-9a-fA-F]+)$ ]]; then RES=$(( 16#${BASH_REMATCH[1]} )); return 0; fi
  if [[ $a =~ ^[dD]:([0-9]+)$ ]]; then RES=${BASH_REMATCH[1]}; return 0; fi
  s=$a
  if [[ ${#s} -eq 10 && ${s:0:1} =~ [-dlcbps] ]]; then
    [[ ${s:0:1} == d ]] && ES_DIR=1
    s=${s:1}
  fi
  RES=$(de_letras "$s") && return 0
  return 1
}

# ------------------------------------------------------------------ umask ---
modo_umask() {
  local arg=${1:-} um
  if [[ -z $arg ]]; then um=$(umask); else um=$arg; fi
  [[ $um =~ ^[0-7]{1,4}$ ]] || error "umask no válida: $um"
  local u=$(( 8#$um & 0777 ))
  local f=$(( 0666 & ~u )) d=$(( 0777 & ~u ))
  printf 'umask %s%s\n\n' "$(printf '%04o' "$u")" "$([[ -z $arg ]] && echo '  (la tuya ahora mismo)')"
  printf '  Ficheros nuevos    666 & ~%03o = %03o   %s\n' "$u" "$f" "$(letras "$f")"
  printf '  Carpetas nuevas    777 & ~%03o = %03o   %s\n' "$u" "$d" "$(letras "$d")"
  # ¿coincidiría una resta "de cabeza" cifra a cifra?
  local a b c r1 r2 r3
  a=$(( 6 - ((u >> 6) & 7) )); b=$(( 6 - ((u >> 3) & 7) )); c=$(( 6 - (u & 7) ))
  r1=$a; r2=$b; r3=$c
  if (( a < 0 || b < 0 || c < 0 )); then
    printf '\n  ⚠  Restar 666 − %03o cifra a cifra da cifras negativas: aquí NO se puede restar.\n' "$u"
    printf '     La umask es una MÁSCARA (se apagan bits), no una resta.\n'
  elif (( r1*64 + r2*8 + r3 != f )); then
    printf '\n  ⚠  Restar 666 − %03o cifra a cifra daría %d%d%d, pero el resultado real es %03o.\n' "$u" "$r1" "$r2" "$r3" "$f"
    printf '     La umask es una MÁSCARA (se apagan bits), no una resta.\n'
  fi
}

# ------------------------------------------------------------------ main ---
main() {
  local args=()
  while (( $# )); do
    case $1 in
      -h|--help|--ayuda) uso; return 0 ;;
      -d|--dir) ES_DIR=1 ;;
      -u|--umask) shift; modo_umask "${1:-}"; return 0 ;;
      --) shift; args+=("$@"); break ;;
      *) args+=("$1") ;;
    esac
    shift
  done
  (( ${#args[@]} )) || { uso; return 0; }

  local res base
  # dos argumentos: modo base + expresión de chmod
  if (( ${#args[@]} == 2 )) && es_expresion "${args[1]}" && [[ ! -e ${args[1]} ]]; then
    if [[ -e ${args[0]} && ! ${args[0]} =~ ^[0-7]{3,4}$ ]]; then
      base=$(( 8#$(stat -c %a -- "${args[0]}") )); [[ -d ${args[0]} ]] && ES_DIR=1
    else
      reconocer "${args[0]}" || error "no entiendo el modo base: ${args[0]}"
      base=$RES
    fi
    res=$(aplicar "$base" "${args[1]}" "$ES_DIR") || error "expresión de chmod no válida: ${args[1]}"
    printf 'Base %s  +  chmod %s\n\n' "$(printf '%03o' "$base")" "${args[1]}"
    mostrar "$res" "$ES_DIR" "Resultado:"
    if [[ ${args[1]} =~ (^|,)[-+=] ]]; then
      printf '\n  (Sin u/g/o/a delante, chmod respeta tu umask actual: %s)\n' "$(umask)"
    fi
    return 0
  fi

  local first=1 a m t dir_save
  for a in "${args[@]}"; do
    (( first )) || printf '\n──────────────────────────────────────────\n\n'
    first=0
    # ¿fichero real?
    if [[ -e $a || -L $a ]] && ! [[ $a =~ ^[0-7]{3,4}$ ]]; then
      m=$(stat -c %a -- "$a"); t=0; [[ -d $a ]] && t=1
      mostrar "$(( 8#$m ))" "$t" "$(stat -c '%F: %n   (dueño %U, grupo %G)' -- "$a")"
      continue
    fi
    dir_save=$ES_DIR
    if reconocer "$a"; then
      mostrar "$RES" "$ES_DIR" "$a"
    elif es_expresion "$a"; then
      res=$(aplicar 0 "$a" "$ES_DIR") || error "expresión de chmod no válida: $a"
      printf 'chmod %s   aplicado a 000\n\n' "$a"
      mostrar "$res" "$ES_DIR" "Resultado:"
    else
```

```
      error "no entiendo '$a' (prueba: permisos -h)"
    fi
    ES_DIR=$dir_save
  done
}

main "$@"
```

Anexo C · `diagnostica`

Explica por qué un usuario puede o no acceder a una ruta. Instálalo igual: `sudo install -m 755 diagnostica /usr/local/bin/`.

```
#!/usr/bin/env bash
# ============================================================================
#  diagnostica · ¿por qué un usuario SÍ o NO puede acceder a una ruta?
#
#  Recorre la ruta tramo a tramo (/, /home, /home/ana, …) aplicando EXACTAMENTE
#  el algoritmo de los permisos de Linux (root → dueño → grupo → otros) y te
#  dice qué tramo bloquea el acceso. Después avisa de lo que esta herramienta
#  NO evalúa (ACL, atributos, montaje, AppArmor/SELinux).
#
#  Para poder mirar carpetas ajenas, ejecútala con sudo:
#      sudo diagnostica -u carla /home/alumno/lab-permisos/datos/secreto.txt
# ============================================================================
LC_ALL=C
export LC_ALL

uso() {
cat <<'EOF'
diagnostica · ¿por qué un usuario SÍ o NO puede acceder a una ruta?

USO
  diagnostica [-u USUARIO] RUTA

  -u USUARIO   mira el acceso de ese usuario (por defecto, tú)
  -h           esta ayuda

EJEMPLOS
  diagnostica /etc/shadow
  sudo diagnostica -u carla /home/alumno/lab-permisos/datos/secreto.txt
  sudo diagnostica -u www-data /var/www/html/index.html

Para ver carpetas que no son tuyas ejecútala con sudo.
EOF
}

error() { printf 'diagnostica: %s\n' "$*" >&2; exit 1; }

usuario=$(id -un)
while getopts "u:h" o; do
  case $o in
    u) usuario=$OPTARG ;;
    h) uso; exit 0 ;;
    *) uso; exit 1 ;;
  esac
done
shift $((OPTIND - 1))
[[ $# -eq 1 ]] || { uso; exit 1; }

id -u "$usuario" >/dev/null 2>&1 || error "el usuario '$usuario' no existe"
uid=$(id -u "$usuario")
gids=" $(id -G "$usuario") "
grupos=$(id -Gn "$usuario")

dest=$(realpath -m -- "$1")
[[ -e $dest || -L $dest ]] || printf '⚠  La ruta %s no existe (o no puedo verla desde aquí; prueba con sudo).\n\n' "$dest"

# --- lista de tramos: /, /a, /a/b, … ----------------------------------------
tramos=("/")
acc=""
IFS=/ read -ra partes <<<"${dest#/}"
for p in "${partes[@]}"; do
  [[ -z $p ]] && continue
  acc+="/$p"
  tramos+=("$acc")
done

ancho=0
for t in "${tramos[@]}"; do (( ${#t} > ancho )) && ancho=${#t}; done

letras_trio() {  # 0..7 → rwx
  local d=$1 s=""
  (( d & 4 )) && s+=r || s+=-
  (( d & 2 )) && s+=w || s+=-
  (( d & 1 )) && s+=x || s+=-
  printf '%s' "$s"
}

printf 'Usuario: %s (uid %s)   Grupos: %s\n' "$usuario" "$uid" "$grupos"
printf 'Ruta:    %s\n\n' "$dest"

n=${#tramos[@]}
bloqueo=""
perm_final_r=0 perm_final_w=0 perm_final_x=0
es_dir_final=0
```

```
tiene_acl=0

printf '  %-*s  %-10s  %-15s  %-6s %-3s  %s\n' "$ancho" "Tramo" "Permisos" "Dueño:Grupo" "Aplica" "rwx" "¿Puede?"
for (( i=0; i<n; i++ )); do
  t=${tramos[i]}
  if ! info=$(stat -c '%a|%u|%g|%U|%G|%A|%F' -- "$t" 2>/dev/null); then
    printf '  %-*s  (no puedo ver este tramo: ejecuta con sudo)\n' "$ancho" "$t"
    [[ -z $bloqueo ]] && bloqueo="$t (no visible desde aquí)"
    continue
  fi
  IFS='|' read -r modo ofu ofg onu ong letras tipo <<<"$info"
  m=$(( 8#$modo ))
  if (( uid == 0 )); then
    rol=root; trio=7
  elif (( uid == ofu )); then
    rol=dueño; trio=$(( (m >> 6) & 7 ))
  elif [[ $gids == *" $ofg "* ]]; then
    rol=grupo; trio=$(( (m >> 3) & 7 ))
  else
    rol=otros; trio=$(( m & 7 ))
  fi
  r=$(( (trio >> 2) & 1 )); w=$(( (trio >> 1) & 1 )); x=$(( trio & 1 ))
  es_dir=0; [[ $tipo == directory ]] && es_dir=1
  # root: ejecutar un fichero exige alguna x
  if (( uid == 0 && es_dir == 0 )); then (( (m & 0111) )) && x=1 || x=0; fi
  [[ $(ls -ld -- "$t" 2>/dev/null | cut -c11) == "+" ]] && tiene_acl=1

  if (( i < n - 1 )); then
    # tramo intermedio: solo importa la x
    if (( x )); then res="✔ puede atravesarlo"; else res="✘ FALTA x para atravesarlo"; [[ -z $bloqueo ]] && bloqueo="$t  ($rol:
$(letras_trio "$trio"))"; fi
  else
    es_dir_final=$es_dir
    perm_final_r=$r; perm_final_w=$w; perm_final_x=$x
    res="(el objeto final: ver abajo)"
  fi
  printf '  %-*s  %-10s  %-15s  %-6s %-3s  %s\n' "$ancho" "$t" "$letras" "$onu:$ong" "$rol" "$(letras_trio "$trio")" "$res"
done

echo
if [[ -n $bloqueo ]]; then
  printf '✘ BLOQUEO en: %s\n' "$bloqueo"
  printf '  Sin poder atravesar ese tramo, %s no llega al objeto: NO puede hacer nada con él.\n' "$usuario"
  printf '  Remedio: dar la x que falta en esa carpeta (chmod o+x / g+x, o añadir al usuario al grupo).\n'
else
  if (( es_dir_final )); then
    a1="listar (r)"; a2="crear/borrar dentro (w)"; a3="entrar (x)"
  else
    a1="leer (r)"; a2="escribir (w)"; a3="ejecutar (x)"
  fi
  mk() { (( $1 )) && printf '✔' || printf '✘'; }
  printf 'Camino libre hasta el objeto. Sobre %s, %s puede:\n' "$dest" "$usuario"
  printf '   %s %s\n   %s %s\n   %s %s\n' "$(mk $perm_final_r)" "$a1" "$(mk $perm_final_w)" "$a2" "$(mk $perm_final_x)" "$a3"
  if (( es_dir_final )) && (( perm_final_w )) && (( ! perm_final_x )); then
    printf '   (w sin x en una carpeta no sirve para nada)\n'
  fi
fi

# --- lo que NO evalúa esta herramienta ----------------------------------------
echo
avisos=()
(( tiene_acl )) && avisos+=("hay una ACL en el camino (un '+' en ls -l): mira con getfacl")
atr=$(lsattr -d -- "$dest" 2>/dev/null | cut -c1-20)
if [[ -n $atr ]]; then
  [[ ${atr:4:1} == i ]] && avisos+=("el objeto es INMUTABLE (chattr +i): nadie lo modifica ni borra, ni root")
  [[ ${atr:5:1} == a ]] && avisos+=("el objeto es SOLO-AÑADIR (chattr +a)")
fi
opts=$(findmnt -T "$dest" -no OPTIONS 2>/dev/null)
if [[ -n $opts ]]; then
  [[ ,$opts, == *,ro,* ]]     && avisos+=("partición montada SOLO LECTURA (ro)")
  [[ ,$opts, == *,noexec,* ]] && avisos+=("partición montada noexec: no se ejecuta nada")
  [[ ,$opts, == *,nosuid,* ]] && avisos+=("partición montada nosuid: SUID/SGID ignorados")
fi
if (( ${#avisos[@]} )); then
  echo "⚠ Ojo: esta herramienta NO aplica esto al veredicto, pero puede cambiarlo:"
  for a in "${avisos[@]}"; do echo "   · $a"; done
else
  echo "Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas."
fi
echo "(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)"
```